

Лабораторная работа 10

Итоговый проект, часть 2. Рефакторинг в ООП и расширение

Цель: преобразовать процедурную программу в объектно-ориентированную, применить принципы ООП (инкапсуляция, наследование, полиморфизм) и добавить новые возможности.

Введение

Вы возьмёте готовое процедурное приложение из Лабораторной работы 9 и перепишите его с использованием классов. Затем расширите функциональность: добавьте поддержку подзадач, сроки выполнения и простую систему напоминаний (через проверку даты).

Теоретическая часть

- Класс `Task` будет содержать атрибуты: `title`, `done`, `category`, `due_date` (опционально), `subtasks` (список подзадач).
- Класс `TaskManager` будет управлять списком задач, загрузкой/сохранением, статистикой и всеми операциями.
- Инкапсуляция: внутренний список задач (`_tasks`) и методы доступа.
- Сохранение в JSON для удобства работы со сложной структурой (подзадачи, даты). Используйте модуль `json`.

Задания

1. Класс `Task`

Создайте класс `Task` с конструктором `__init__(self, title, category="Общее", due_date=None)`.

Атрибуты: `title`, `done=False`, `category`, `due_date` (строка в формате ГГГГ-ММ-ДД или `None`), `subtasks` (список словарей `{title: str, done: bool}`). Добавьте метод `__str__` для красивого отображения.

2. Класс `TaskManager` – загрузка и сохранение

Создайте класс `TaskManager` с атрибутом `_tasks` (список объектов `Task`). Реализуйте методы `load_from_json(filename)` и `save_to_json(filename)` с использованием модуля `json`. Для сериализации объектов `Task` придётся написать вспомогательную функцию или метод `to_dict()` в классе `Task` и `from_dict()`.

3. Методы управления

В `TaskManager` добавьте методы:

- `add_task(title, category, due_date=None)`
- `remove_task(index)`
- `mark_done(index)`
- `get_task(index)`

Все методы должны проверять корректность индекса и выбрасывать исключение при неверном индексе.

4. Отображение и фильтрация

Реализуйте методы:

- `list_all()` – вывод всех задач
- `list_by_category(category)`
- `list_pending()` – только невыполненные
- `list_overdue()` – задачи, у которых `due_date` раньше сегодняшней даты (используйте `datetime.date.today()`).

5. Подзадачи

Добавьте в меню возможность:

- добавить подзадачу к существующей задаче (метод `add_subtask(task_index, title)`)
- отметить подзадачу выполненной

Методы должны быть в `TaskManager`, но можно делегировать в `Task`.

6. Статистика

Реализуйте метод `get_stats()` в `TaskManager`, который возвращает словарь со статистикой: всего задач, выполнено, просрочено, по категориям.

7. Поиск

Добавьте метод `search(query)`, возвращающий список задач, в названии которых встречается `query` (регистронезависимо), включая поиск по названиям подзадач.

8. Интерактивное меню (ООП-версия)

Напишите функцию `main()`, которая создаёт объект `TaskManager`, загружает данные и запускает цикл меню с теми же пунктами, что и в первой части, но работающими через методы объекта.

9. Наследование для разных типов задач

Создайте подкласс `PriorityTask(Task)`, добавляющий поле `priority` (целое число).

Переопределите `__str__` для отображения приоритета. Интегрируйте его в менеджер: при добавлении задачи с приоритетом создавайте `PriorityTask`.

10. Экспорт в календарь

Реализуйте метод `export_ics(filename)` в `TaskManager`, который создаёт простой iCalendar-файл (формат `.ics`) с задачами, у которых установлен срок. Изучите формат iCalendar (`BEGIN:VCALENDAR, VEVENT`) и напишите минимальную реализацию.

11. Резервное копирование

При сохранении создавайте резервную копию предыдущего файла (переименование с добавлением даты). Обработайте ошибки файловой системы.

12. Настройки

Создайте класс `Settings` с атрибутами (например, `auto_save, language`), который загружается из JSON-файла. Используйте его в `TaskManager` для настройки поведения.

13. Логирование через декоратор

Напишите декоратор `@log_call`, который записывает в файл `calls.log` имя вызванного метода и время вызова. Примените его к основным методам `TaskManager`.

14. Тестирование

Напишите несколько простых тестов (`assert`) для проверки методов `Task` и `TaskManager` (без использования библиотек, просто проверки при запуске).

15. Документация

Добавьте docstring ко всем классам и методам. Сгенерируйте текстовый файл README.txt с кратким описанием программы и инструкцией пользователя.

Контрольные вопросы

1. Какие преимущества дало применение ООП по сравнению с процедурной версией?
2. Для чего нужен метод `__str__` в классах?
3. Как работает наследование в вашем проекте?
4. Зачем используется модуль `json` вместо самодельного CSV?
5. Как вы обрабатываете ошибки в методах класса?

Выводы

Вы создали полноценное приложение-трекер задач, которое можно развивать дальше, и закрепили на практике все ключевые концепции Python.